Introducción a los Algoritmos y Flujo de Datos
Facultad de Ingeniería
Universidad de San Carlos de Guatemala

PROYECTO 2

CONVERSOR DE SISTEMAS NUMÉRICOS EN PYTHON

Raúl Estuardo Toca Urbina
202601353

Introducción

El proyecto consiste en un programa desarrollado en Python 3 que convierte números entre los sistemas binario, octal, decimal y hexadecimal. El usuario puede seleccionar las bases, escribir un número y observar tanto el resultado como el procedimiento utilizado.

Análisis y descomposición del problema

El programa se divide en cinco necesidades principales: validar los datos ingresados, realizar las conversiones, mostrar los menús, administrar un historial temporal y crear ejercicios de práctica. Separar esas tareas facilita la comprensión del código y evita repetir funciones.

Flujo general: ejecutar main.py → mostrar menú principal → seleccionar conversión, historial o práctica → realizar la operación → regresar al menú o salir.

Arquitectura y modularización

main.py

Es el archivo inicial del programa. Llama a la función iniciar_programa() y controla una salida por interrupción.

consola.py

Muestra los menús con print() e input(), selecciona las bases, solicita el número, presenta el resultado y registra la operación.

conversiones.py

Contiene los algoritmos a_decimal(), desde_decimal() y convertir(), encargados de las doce combinaciones.

validaciones.py

Comprueba que la entrada corresponda a la base seleccionada, que no esté vacía y que no supere los 128 dígitos.

historial.py

Guarda operaciones válidas en una lista y permite consultar, eliminar registros y vaciarla.

practica.py

Genera ejercicios aleatorios, valida la respuesta y calcula aciertos, respondidos y porcentaje.

Estructuras de datos

Se emplean diccionarios, cadenas de texto y listas. El historial almacena únicamente las conversiones válidas realizadas desde el menú principal; los ejercicios de práctica no se incorporan a esa lista.

Estructura	Uso
BASES	Diccionario que relaciona Binario, Octal, Decimal y Hexadecimal con 2, 8, 10 y 16.
DIGITOS	Cadena 0123456789ABCDEF usada para obtener o identificar dígitos.
historial	Lista de diccionarios con entrada, origen, destino y resultado.
puntuacion	Diccionario que conserva aciertos y ejercicios respondidos durante la ejecución.
pasos	Lista de cadenas para mostrar cómo se realizó una conversión.

Validación de entradas

El programa admite únicamente enteros no negativos, con hasta 128 dígitos escritos en la base de origen. Se rechazan las entradas vacías, los signos, los espacios, los caracteres fuera de base y los prefijos como 0x. Para hexadecimal se aceptan letras A a F en mayúsculas o minúsculas.

La función validar_numero() devuelve el texto normalizado cuando es correcto o produce un mensaje de error cuando no cumple las reglas. Esta comprobación ocurre antes de llamar a convertir().

Algoritmos de conversión

Para convertir desde una base distinta de decimal, a_decimal() aplica notación posicional: obtiene el valor de cada carácter, lo multiplica por la base elevada a su posición y suma los aportes.

Para convertir desde decimal, desde_decimal() utiliza divisiones enteras sucesivas. Guarda el residuo de cada división, representa los residuos como dígitos y los coloca de derecha a izquierda.

Cuando la base de origen y la de destino son distintas de decimal, primero se obtiene el equivalente decimal y después se calcula el número en la nueva base.

Relación entre conversiones y funciones

Conversión	Procedimiento utilizado
Decimal a Binario	desde_decimal(numero, base destino)
Decimal a Octal	desde_decimal(numero, base destino)
Decimal a Hexadecimal	desde_decimal(numero, base destino)
Binario a Decimal	a_decimal(texto, base origen)
Binario a Octal	a_decimal() y después desde_decimal()
Binario a Hexadecimal	a_decimal() y después desde_decimal()
Octal a Decimal	a_decimal(texto, base origen)
Octal a Binario	a_decimal() y después desde_decimal()
Octal a Hexadecimal	a_decimal() y después desde_decimal()
Hexadecimal a Decimal	a_decimal(texto, base origen)
Hexadecimal a Binario	a_decimal() y después desde_decimal()
Hexadecimal a Octal	a_decimal() y después desde_decimal()

Procedimiento paso a paso

La función convertir() devuelve dos elementos: el resultado y una lista con el desarrollo de la operación. Esa lista se recorre desde consola.py y se imprime en pantalla. Así se puede visualizar cada división, residuo o aporte de la notación posicional.

Historial de conversiones

Cada conversión correcta crea un registro con el número escrito, la base de origen, la base de destino y el resultado. El usuario puede consultar la lista, eliminar un registro mediante su número o vaciarla por completo. El contenido no se guarda después de cerrar el programa.

Modo de práctica

El modo de práctica utiliza random para generar un valor decimal entre 0 y 255, selecciona dos bases diferentes y presenta un ejercicio. La respuesta se valida antes de calificar. El porcentaje se calcula dividiendo los aciertos entre los ejercicios respondidos y multiplicando por 100; si todavía no hay respuestas, se muestra 0.0%.

Fragmentos de la implementación

Notación posicional:

```
total = 0
for posicion, caracter in enumerate(reversed(texto_validado)):
    valor_digito = DIGITOS.index(caracter)
    aporte = valor_digito * (base ** posicion)
    total += aporte
```

Divisiones sucesivas:

```
while actual > 0:
    cociente = actual // base
    residuo = actual % base
    digito = DIGITOS[residuo]
    resultado = digito + resultado
    actual = cociente
```

Registro en historial:

```
historial.append({
    "numero": numero, "origen": origen,
    "destino": destino, "resultado": resultado
})
```

Cálculo de puntuación:

```
if puntuacion["respondidos"] == 0:
    return 0.0
return puntuacion["aciertos"] * 100 / puntuacion["respondidos"]
```

La lógica de las cuatro funciones anteriores se encuentra en conversiones.py, historial.py y practica.py, respectivamente.

Diagramas de flujo de las doce conversiones

A continuación se incluyen los diagramas de flujo realizados a mano. Cada figura corresponde a una conversión y se presenta en una página independiente, siguiendo el orden del proyecto.

Los diagramas se conservan tal como fueron enviados, sin redibujarlos ni modificar sus elementos.

Figura 1. Decimal a Binario

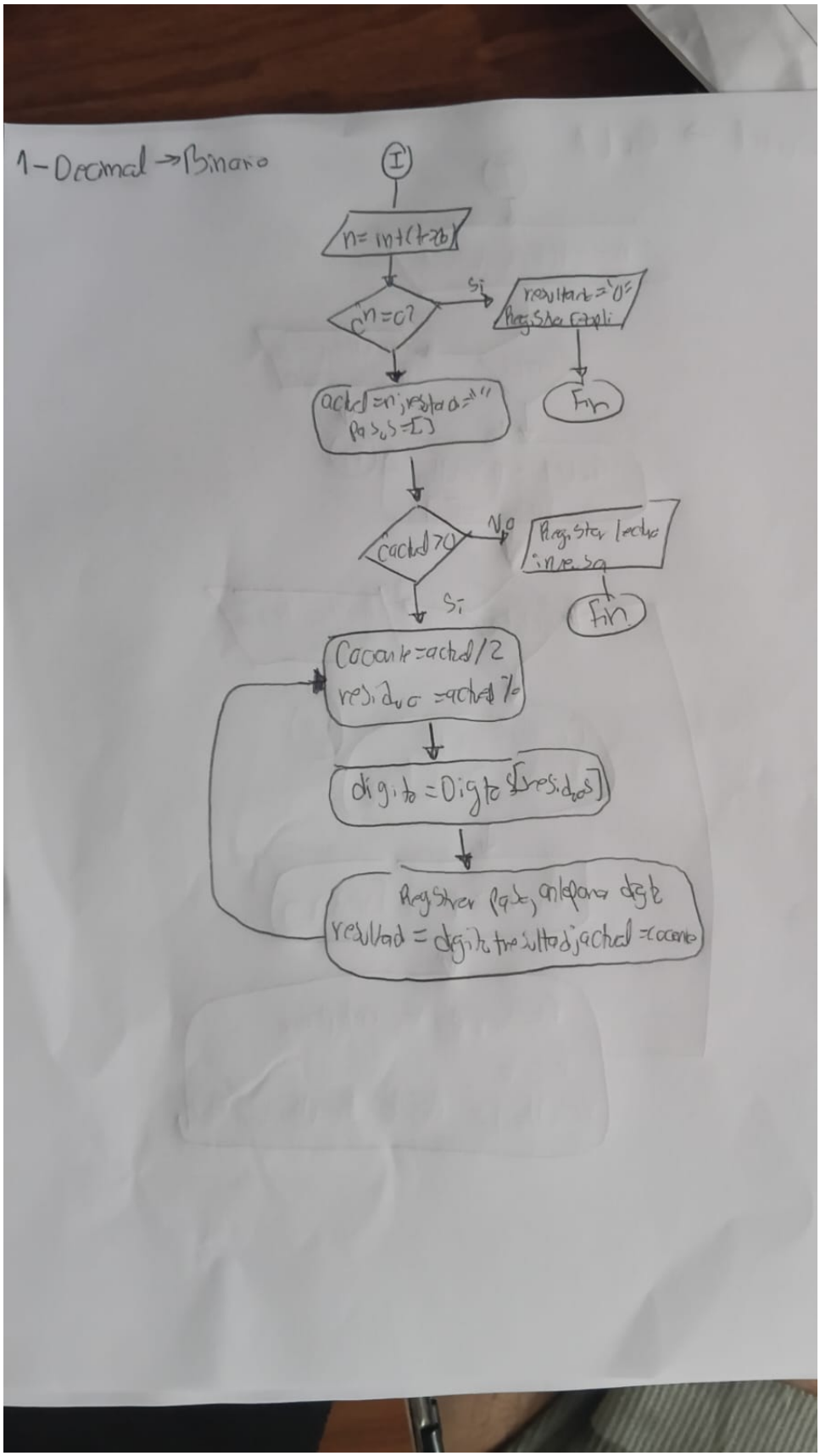


Figura 2. Decimal a Octal

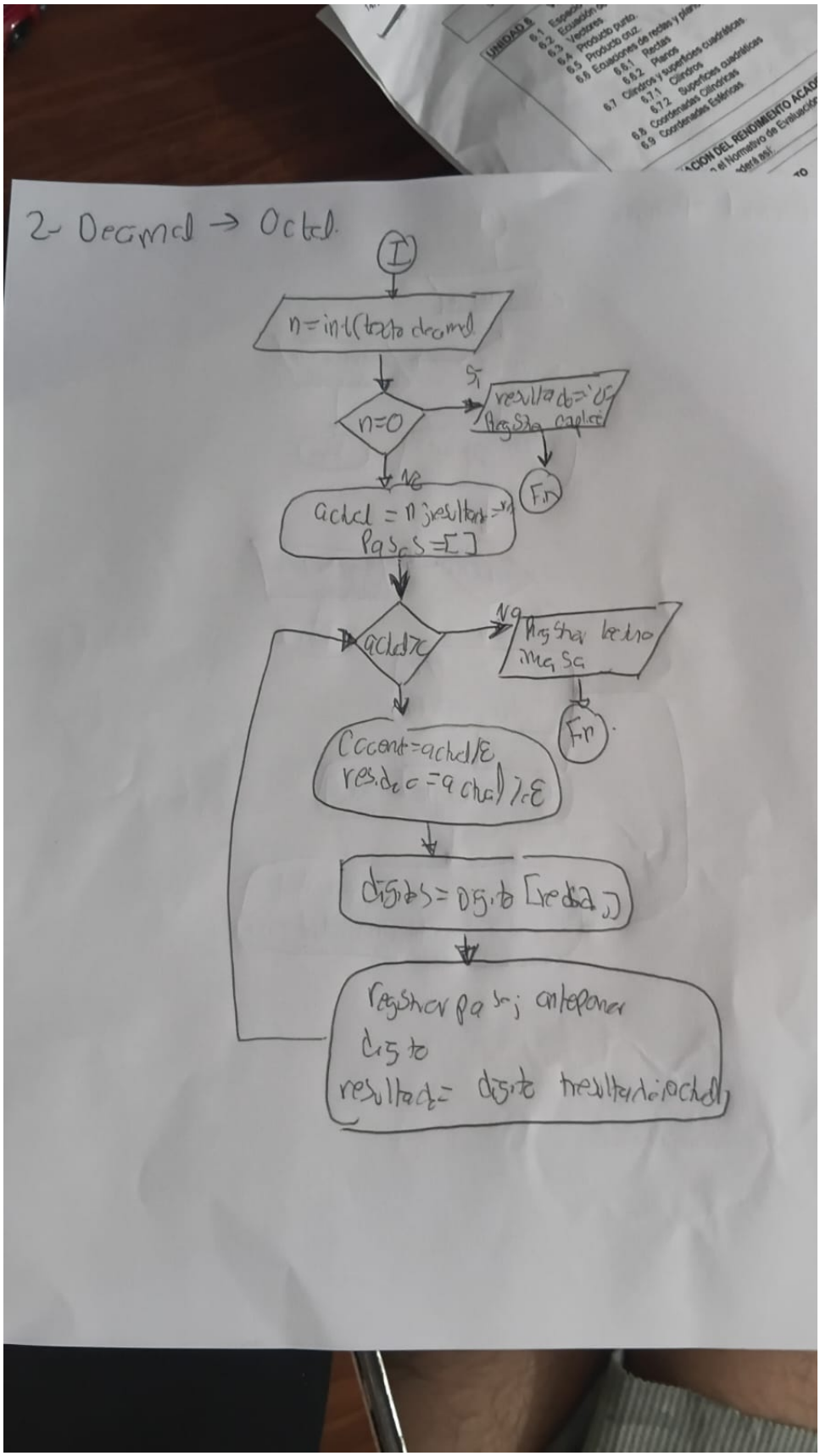


Figura 3. Decimal a Hexadecimal

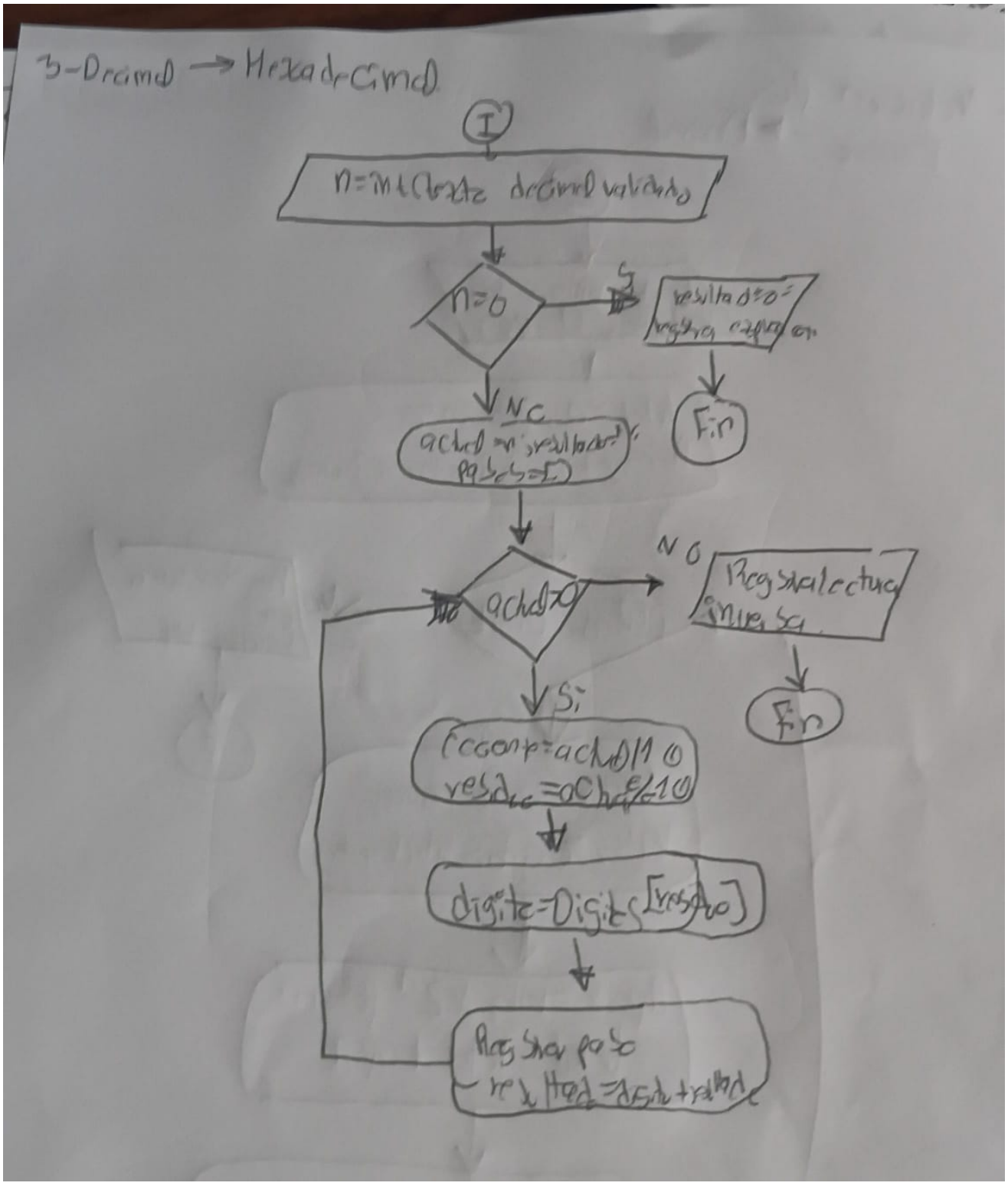


Figura 4. Binario a Decimal

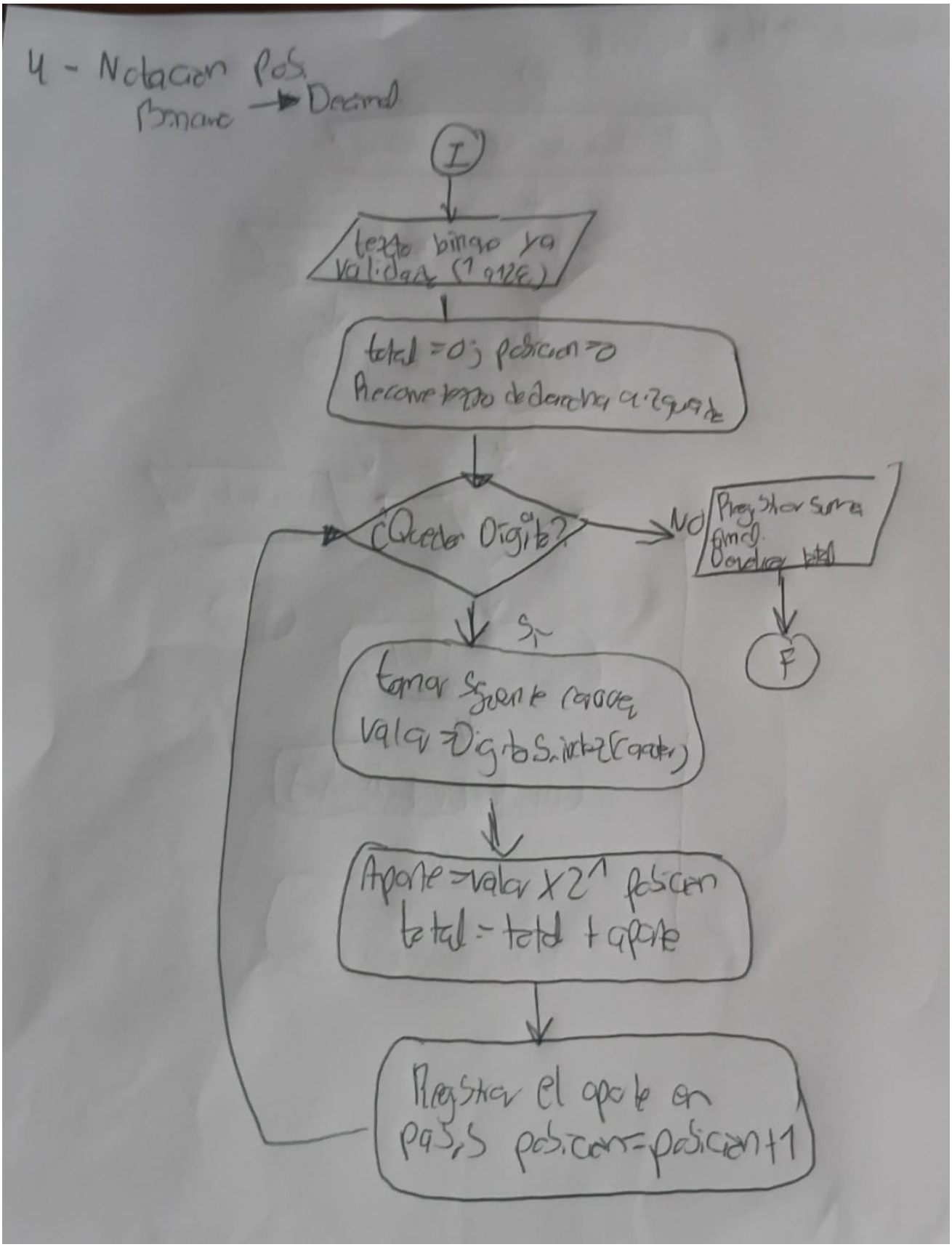


Figura 5. Binario a Octal

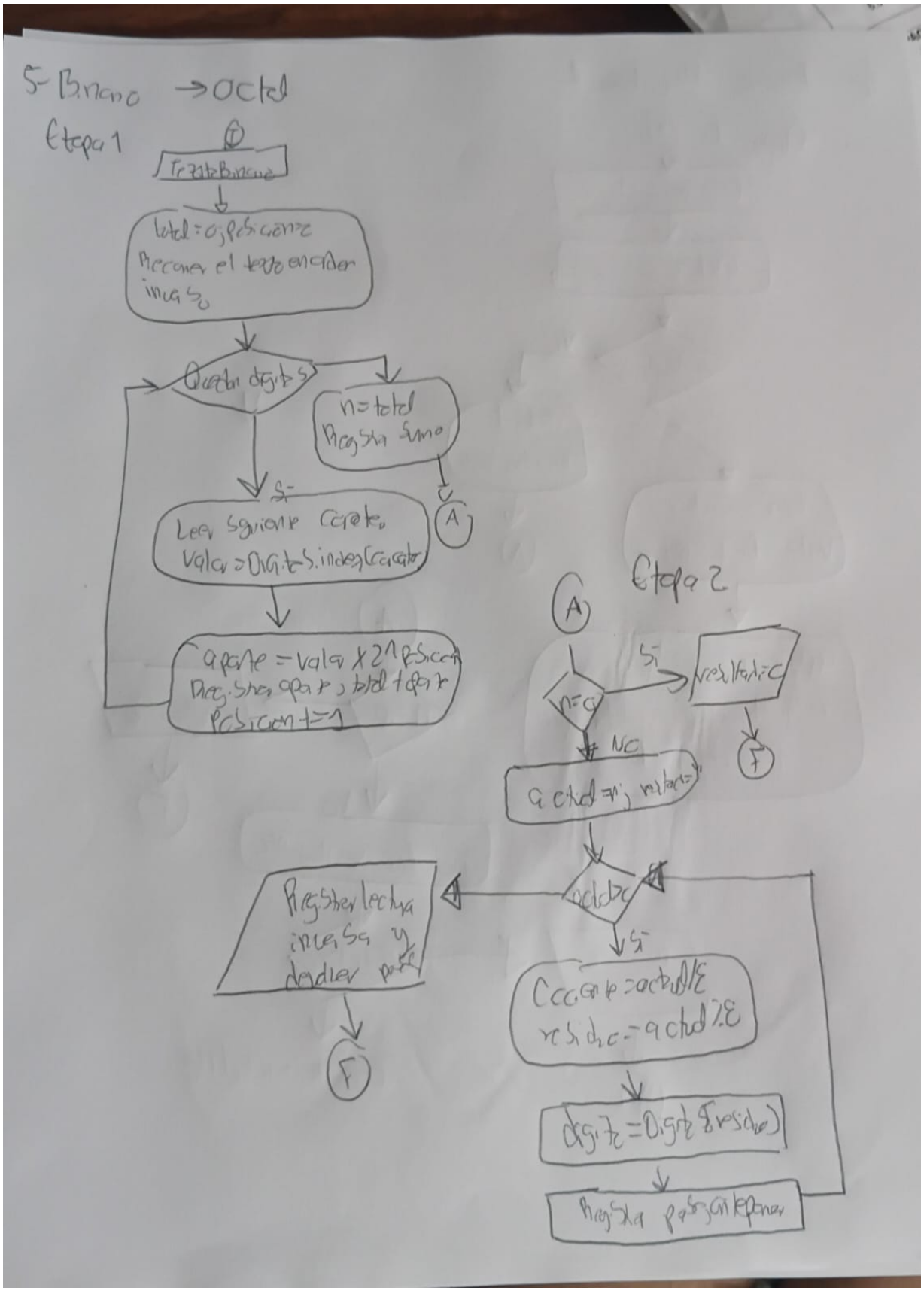


Figura 6. Binario a Hexadecimal

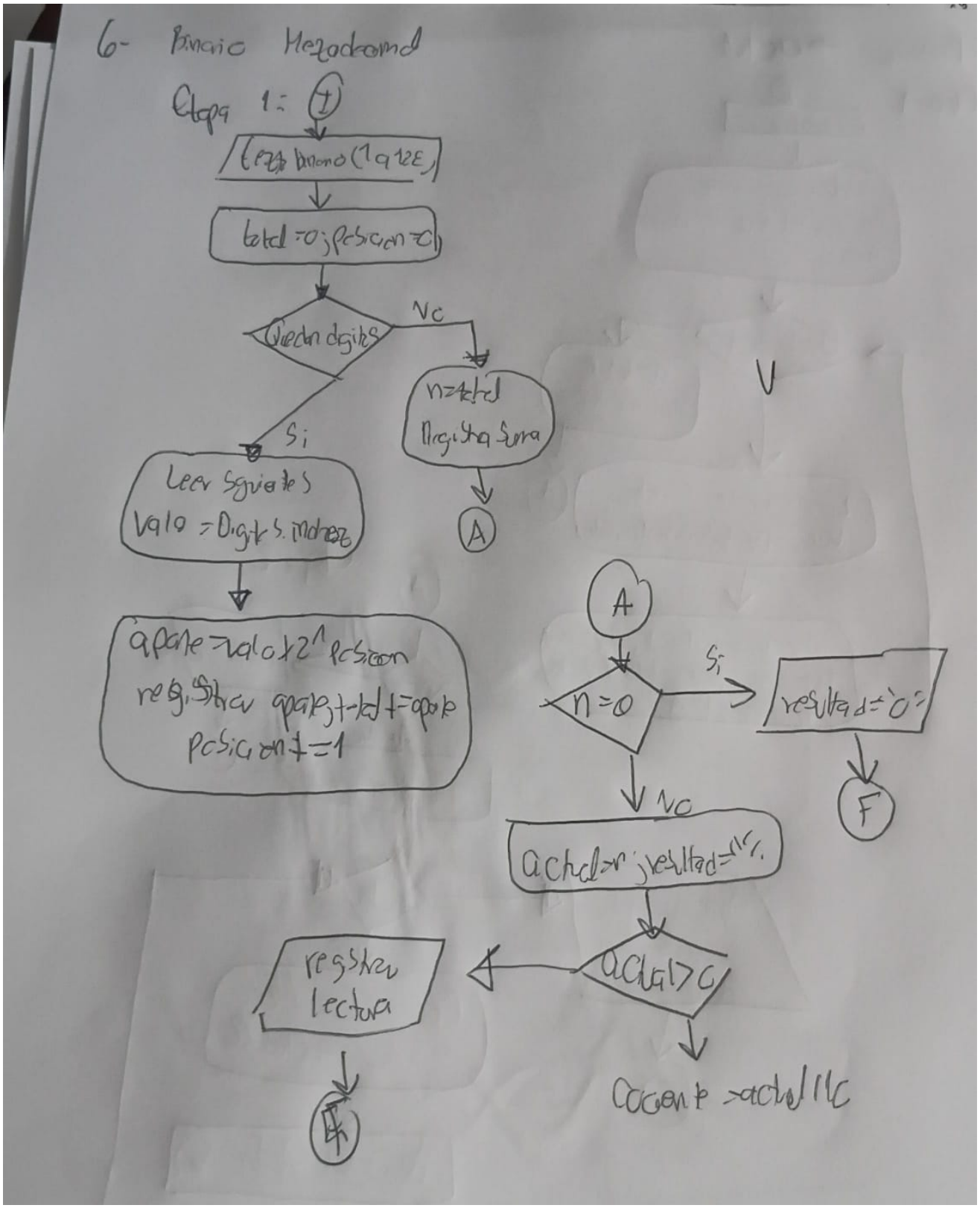


Figura 7. Octal a Decimal

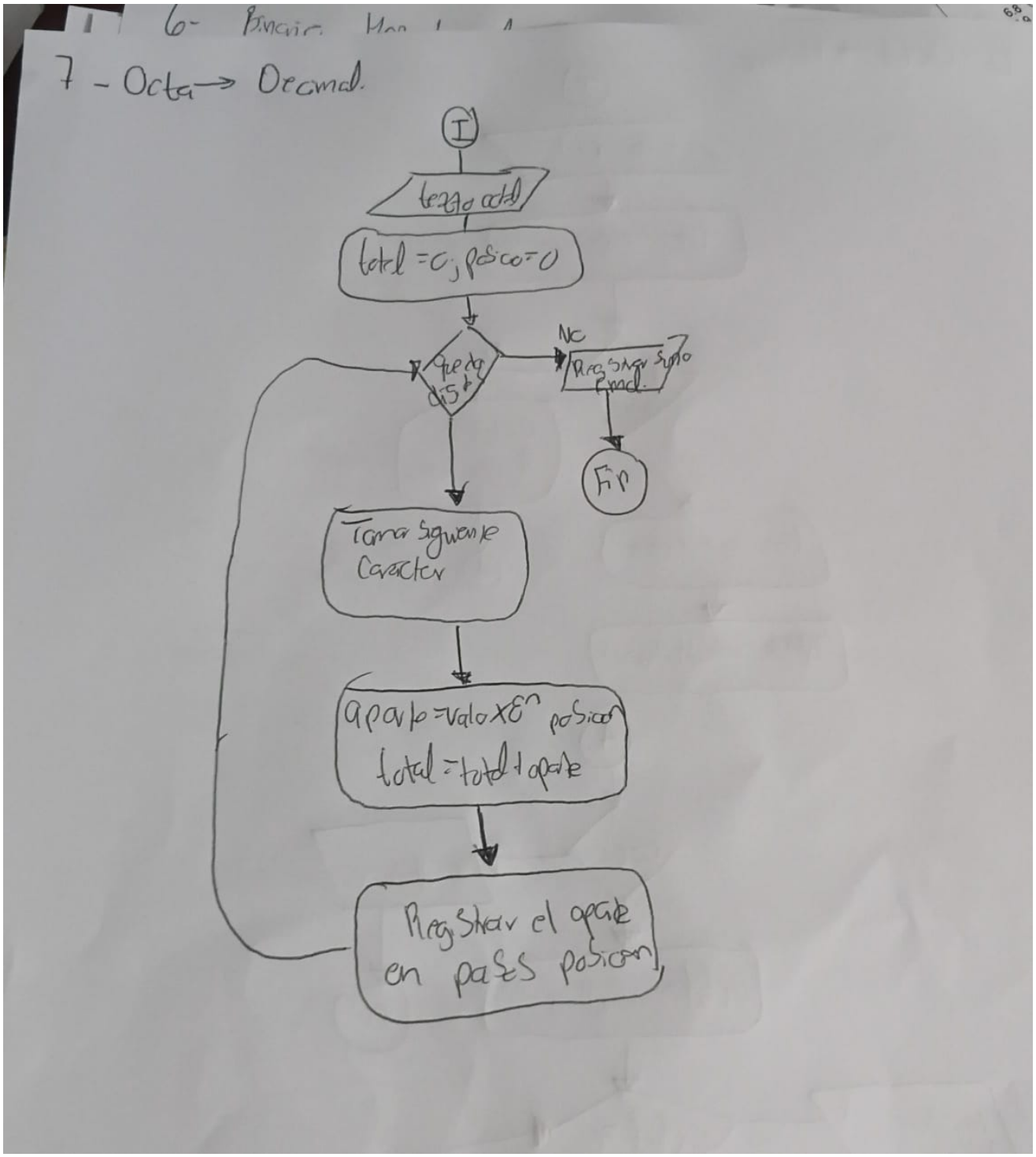


Figura 8. Octal a Binario

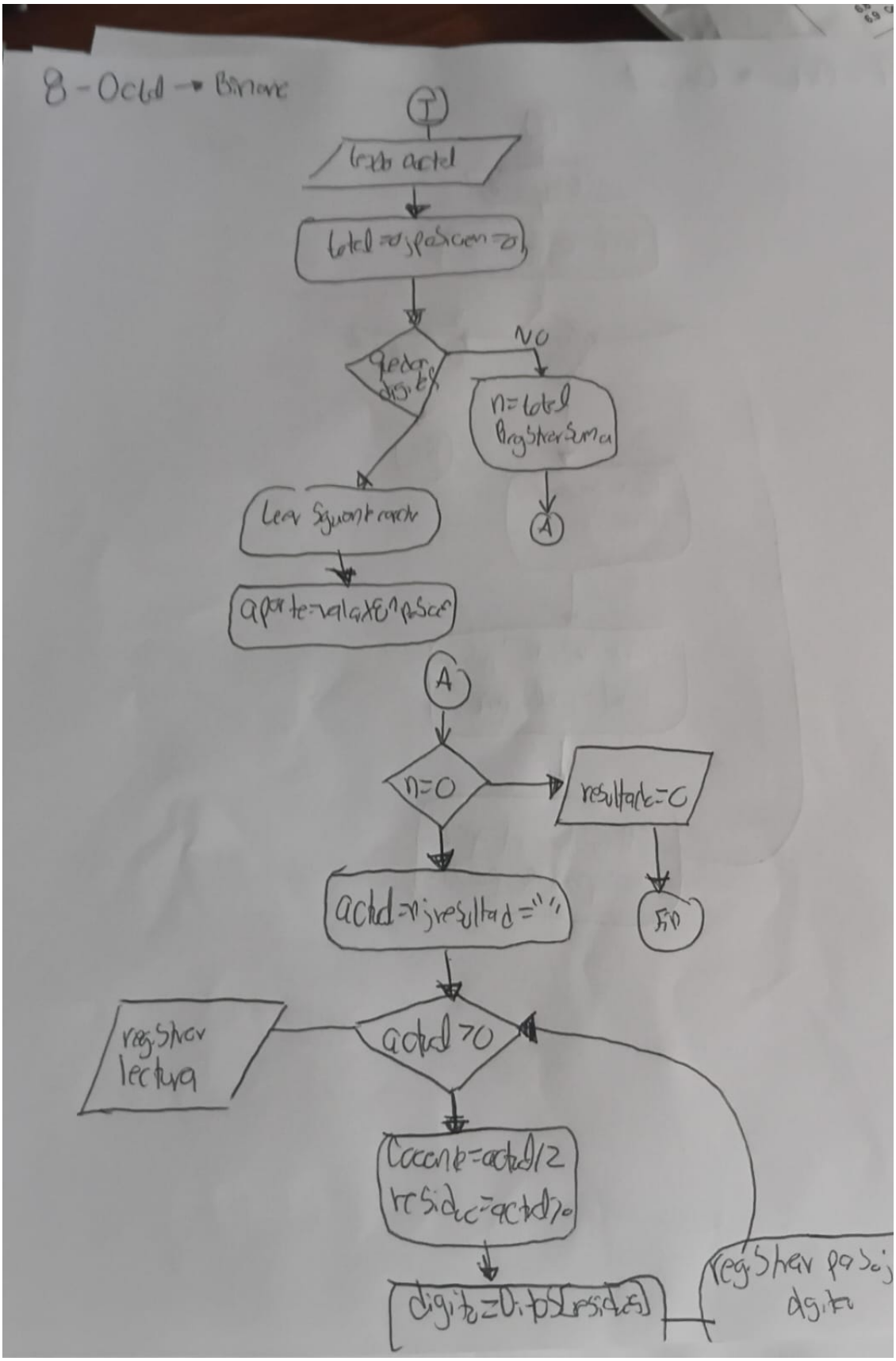


Figura 9. Octal a Hexadecimal

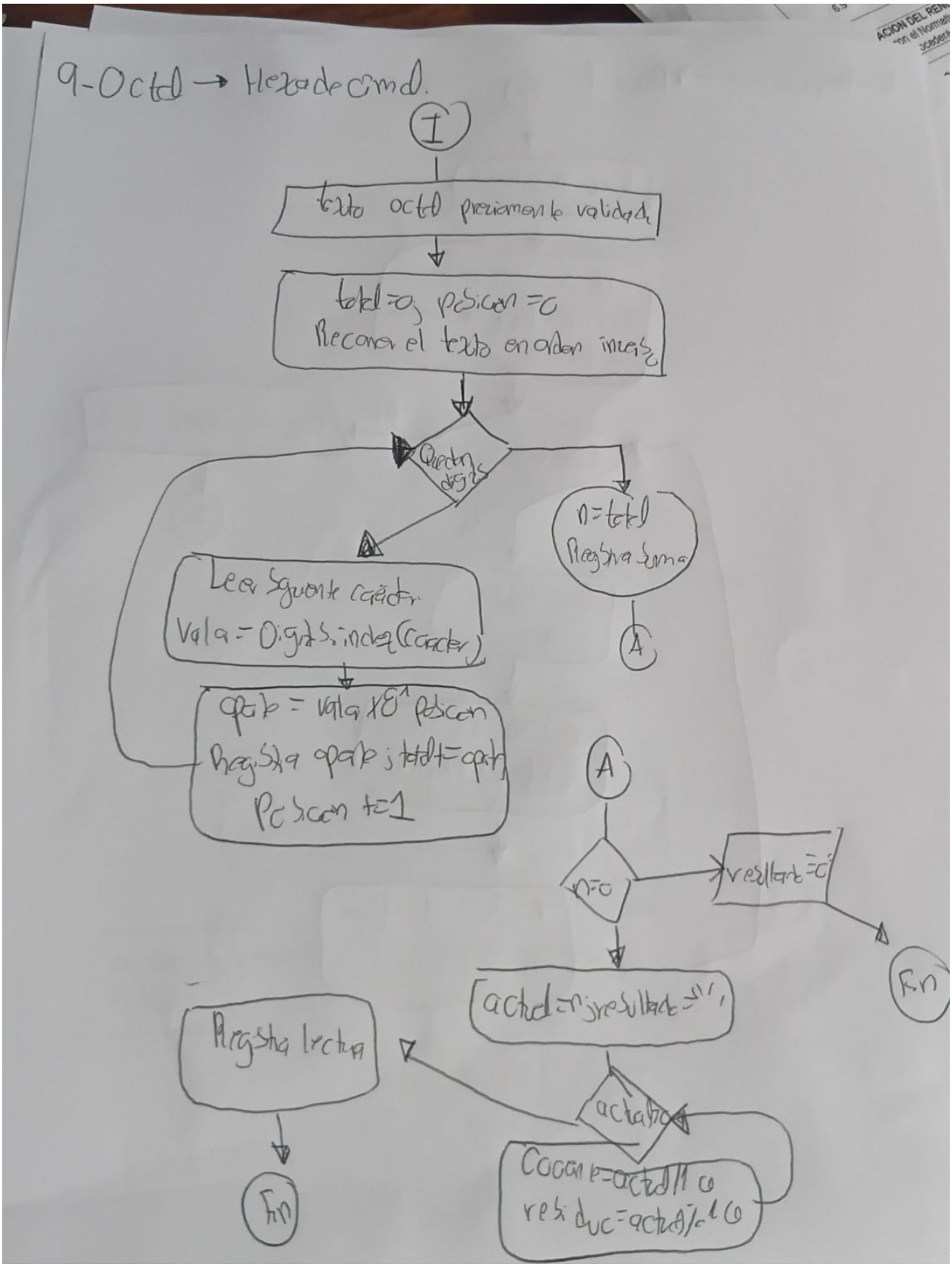


Figura 10. Hexadecimal a Decimal

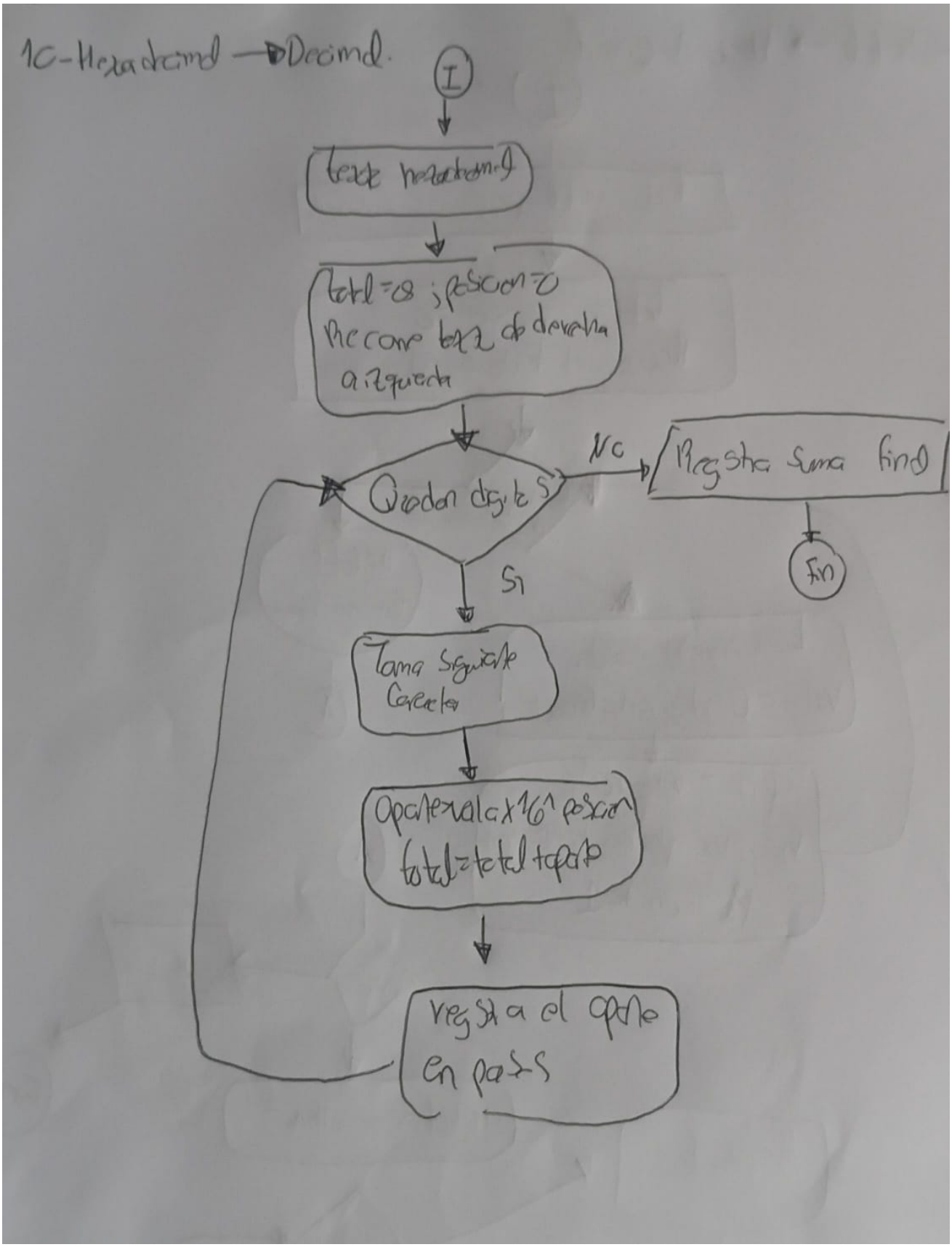


Figura 11. Hexadecimal a Binario

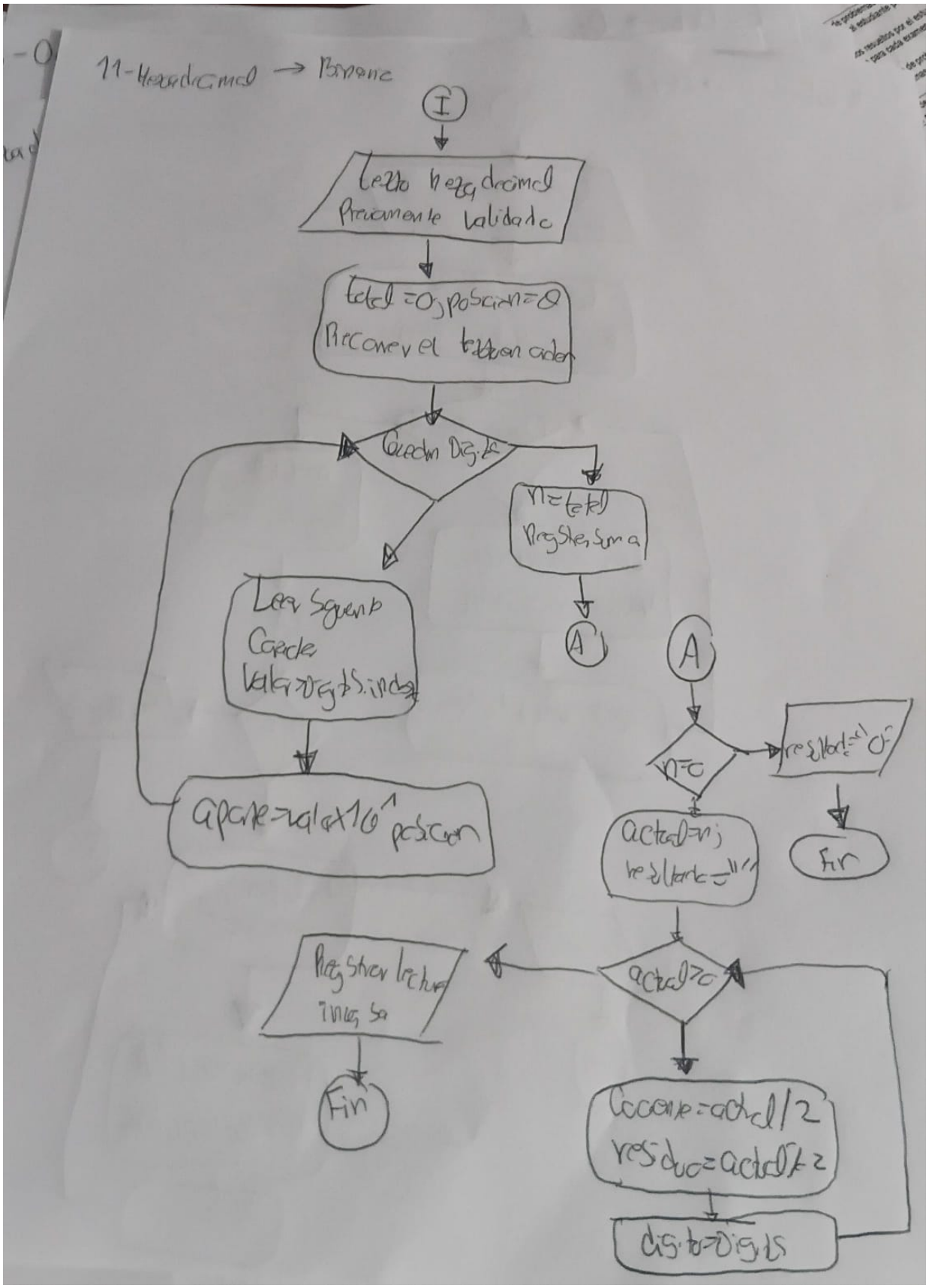
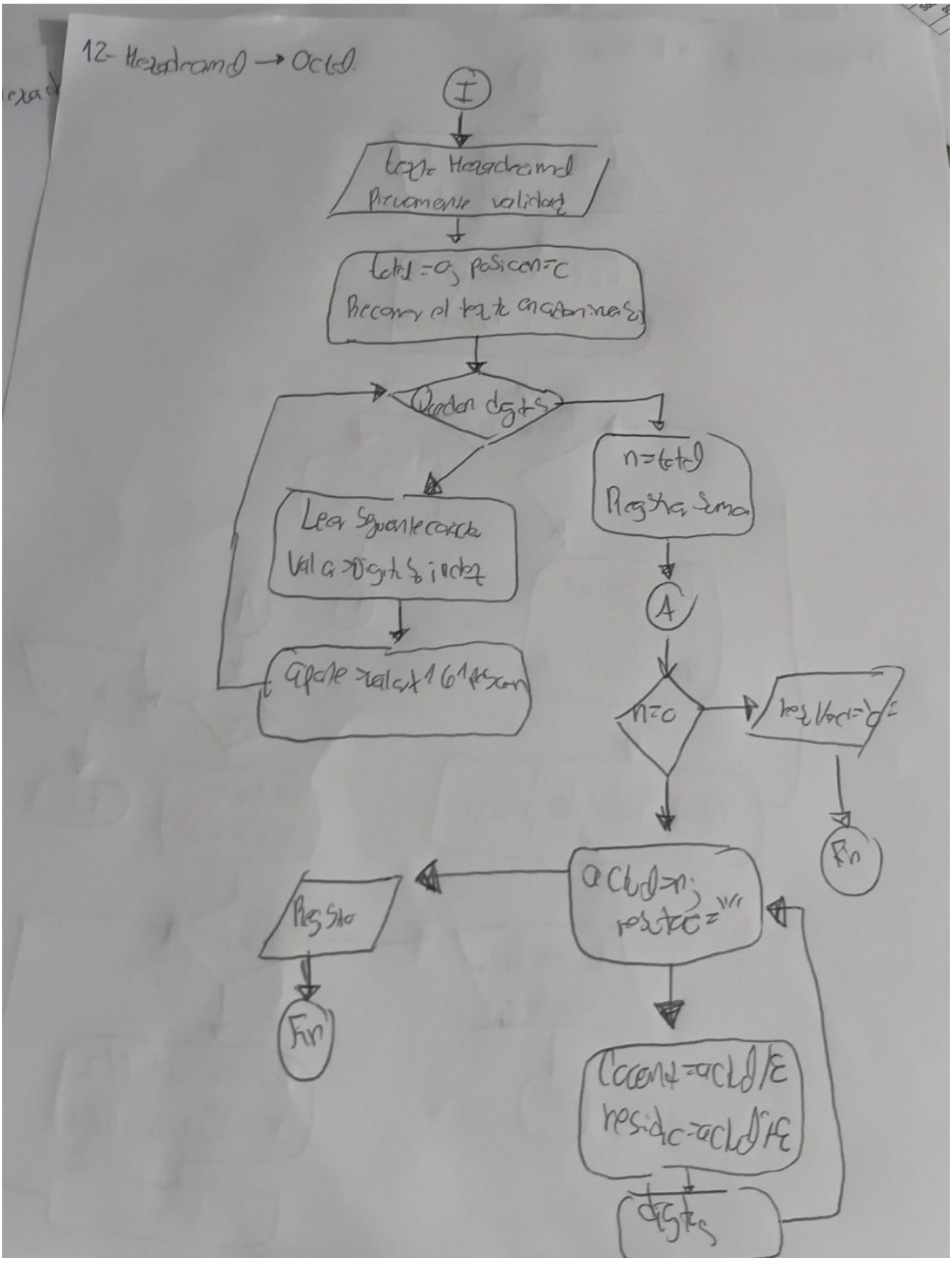


Figura 12. Hexadecimal a Octal



Capturas del funcionamiento

Las siguientes capturas son de la aplicación en ejecución y muestran el resultado de dos algoritmos y la comprobación de entradas.

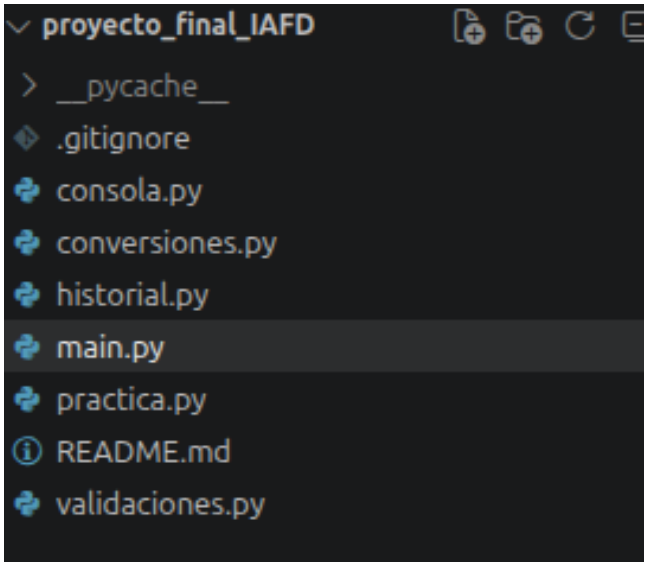


Figura 13. Archivos fuente del proyecto en VS Code.

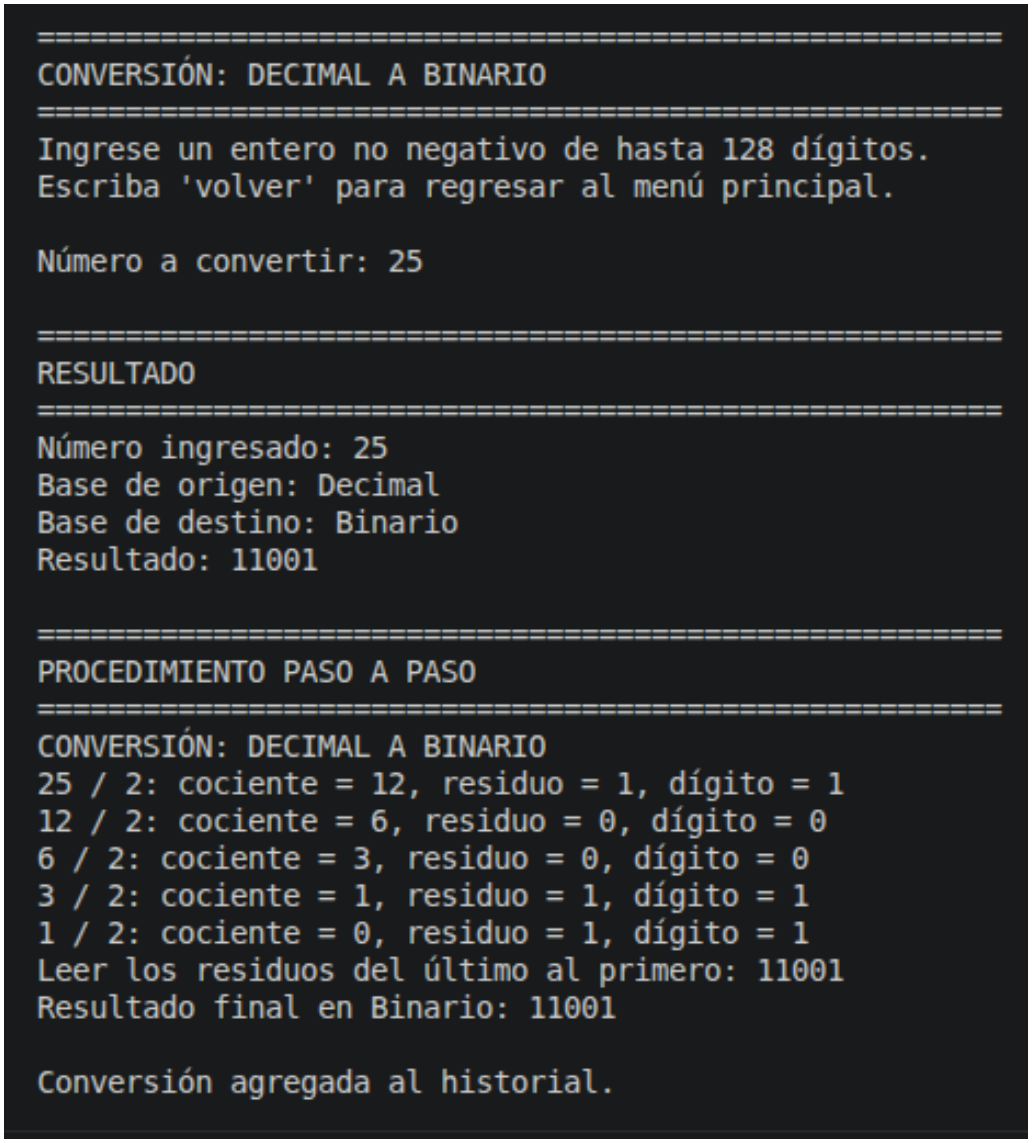


Figura 14. Conversión de 25 decimal a binario y procedimiento paso a paso.

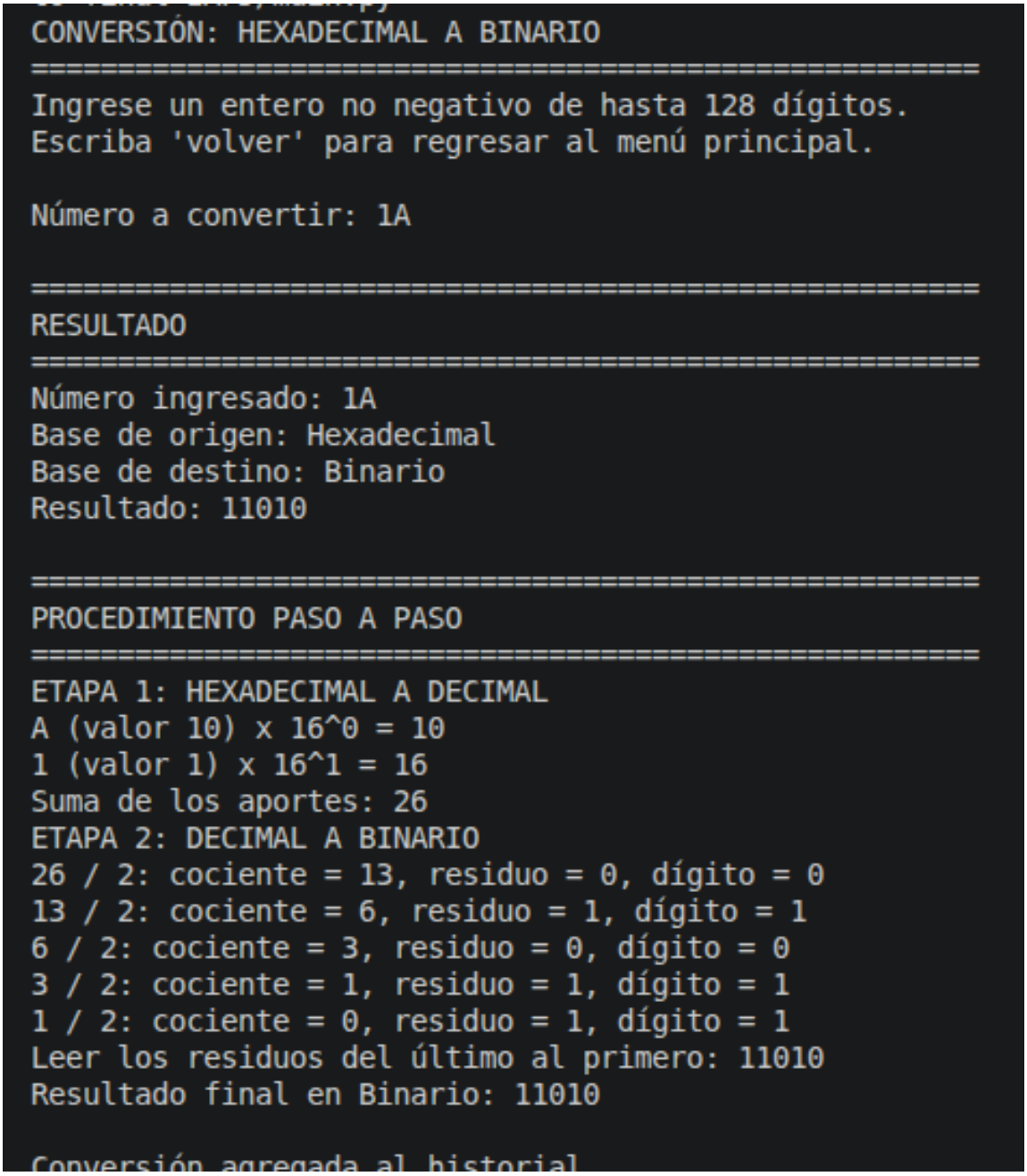


Figura 15. Conversión hexadecimal a binario utilizando decimal como base intermedia.

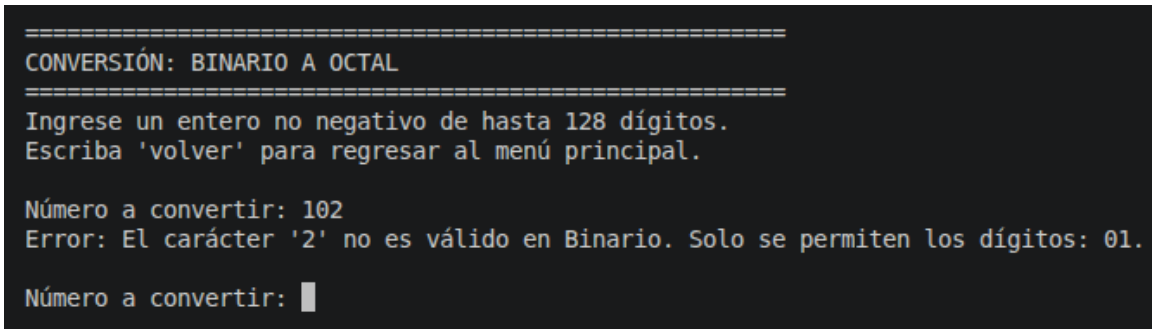


Figura 16. Mensaje mostrado cuando se introduce un dígito no válido.

Pruebas y observaciones

En las capturas se observa el resultado 11001 para el número decimal 25, el resultado 11010 para el hexadecimal 1A, el rechazo del carácter 2 en una entrada binaria y el registro de conversiones en el historial.

También se probaron los menús del historial y la práctica. Los resultados de las pruebas automáticas corresponden a la versión de código utilizada para construir este manual; antes de entregar debe comprobarse que los archivos de la computadora sean los mismos.

Conclusión

La separación por archivos permite ejecutar las doce conversiones, validar los datos, consultar el procedimiento, conservar un historial temporal y practicar con ejercicios. Las funciones de notación posicional y divisiones sucesivas evitan utilizar conversiones automáticas de Python.